A* Algorithm Programs

1

```
graph = {
   'A': [('B', 1), ('C', 3)],
   'B': [('D', 2), ('E', 4)],
   'C': [('F', 2), ('G', 5)],
   'D': [], 'E': [], 'F': [], 'G': []
}


h = {'A': 5, 'B': 4, 'C': 3,
    'D': 3, 'E': 2, 'F': 2, 'G': 0}


def astar(start, goal):
   open_list = [(h[start], 0, start, [start])]
   visited = set()

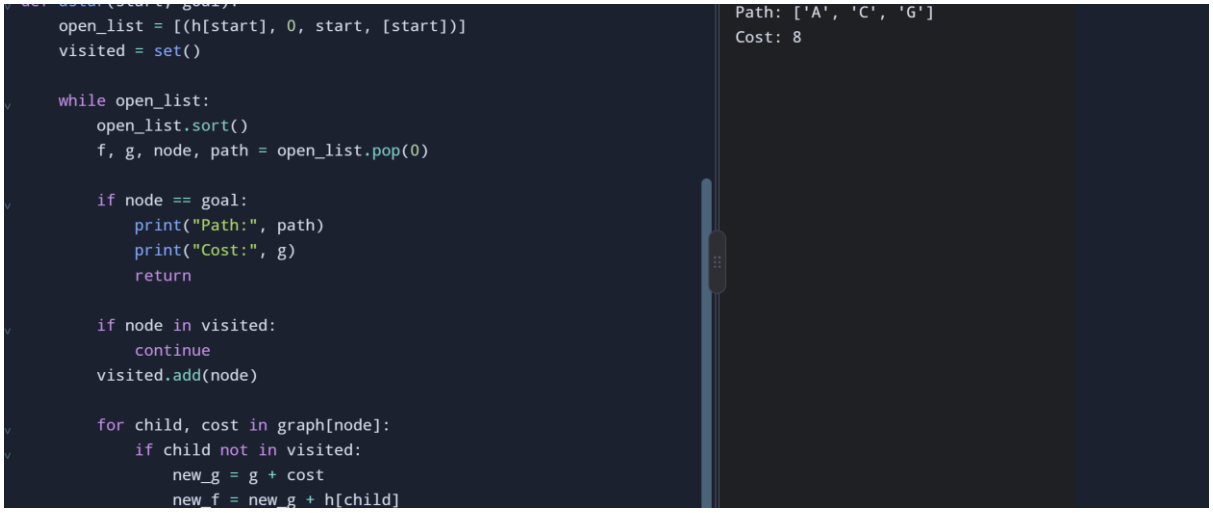

   while open_list:
      open_list.sort()
      f, g, node, path = open_list.pop(0)


      if node == goal:
         print("Path:", path)
         print("Cost:", g)
         return


      if node in visited:
         continue
      visited.add(node)
```

```
        for child, cost in graph[node]:
            if child not in visited:
                new_g = g + cost
                new_f = new_g + h[child]
                open_list.append((new_f, new_g, child, path + [child]))
```

astar('A', 'G')



2

```
graph = {
   'S': [('A', 1), ('B', 4)],
   'A': [('C', 2), ('D', 5)],
   'B': [('D', 1)],
   'C': [('G', 3)],
   'D': [('G', 2)],
   'G': []
}
```

```
h = {'S': 5, 'A': 4, 'B': 3,
    'C': 2, 'D': 1, 'G': 0}
```

```
def astar(start, goal):
    open_list = [(h[start], 0, start, [start])]
    visited = set()

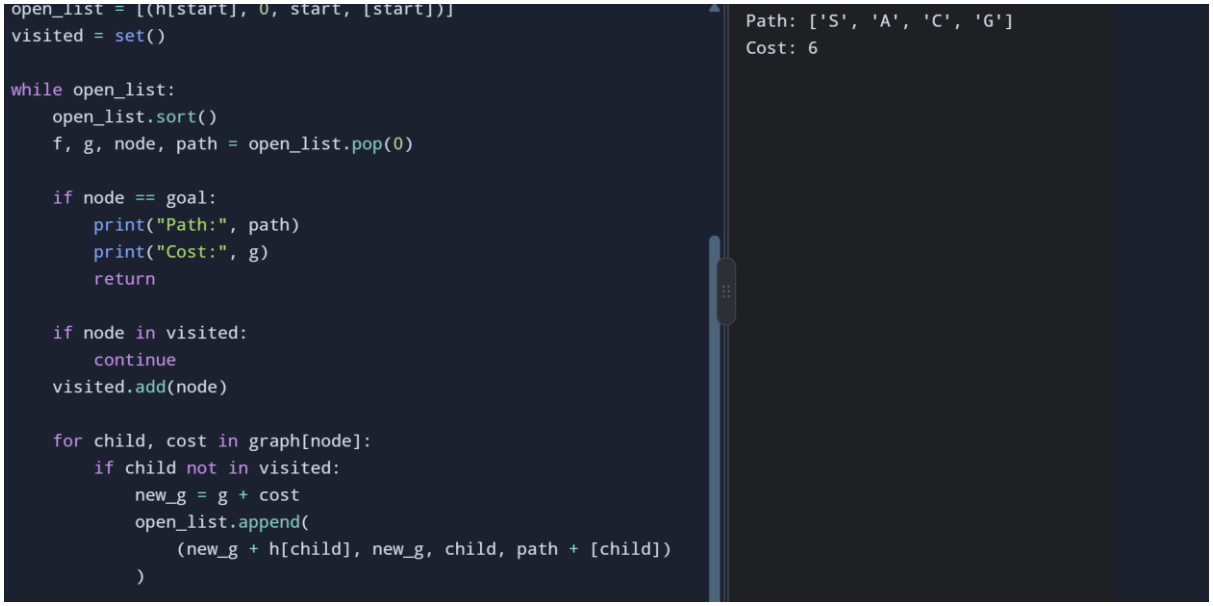
    while open_list:
        open_list.sort()
        f, g, node, path = open_list.pop(0)

        if node == goal:
            print("Path:", path)
            print("Cost:", g)
            return

        if node in visited:
            continue
        visited.add(node)

        for child, cost in graph[node]:
            if child not in visited:
                new_g = g + cost
                open_list.append(
                    (new_g + h[child], new_g, child, path + [child])
                )

astar('S', 'G')
```



```
open_list = [(h[start], 0, start, [start])]
visited = set()

while open_list:
    open_list.sort()
    f, g, node, path = open_list.pop(0)

    if node == goal:
        print("Path:", path)
        print("Cost:", g)
        return

    if node in visited:
        continue
    visited.add(node)

    for child, cost in graph[node]:
        if child not in visited:
            new_g = g + cost
            open_list.append(
                (new_g + h[child], new_g, child, path + [child])
            )
```

```
Path: ['S', 'A', 'C', 'G']
Cost: 6
```

3

```
graph = {
    'A': [('B', 2), ('C', 4)],
    'B': [('D', 3), ('E', 2)],
    'C': [('F', 3)],
    'D': [('F', 4)],
    'E': [('F', 1)],
    'F': []
}


h = {'A': 5, 'B': 4, 'C': 3,
     'D': 2, 'E': 1, 'F': 0}


def astar(start, goal):
    open_list = [(h[start], 0, start, [start])]
    visited = set()


    while open_list:
```

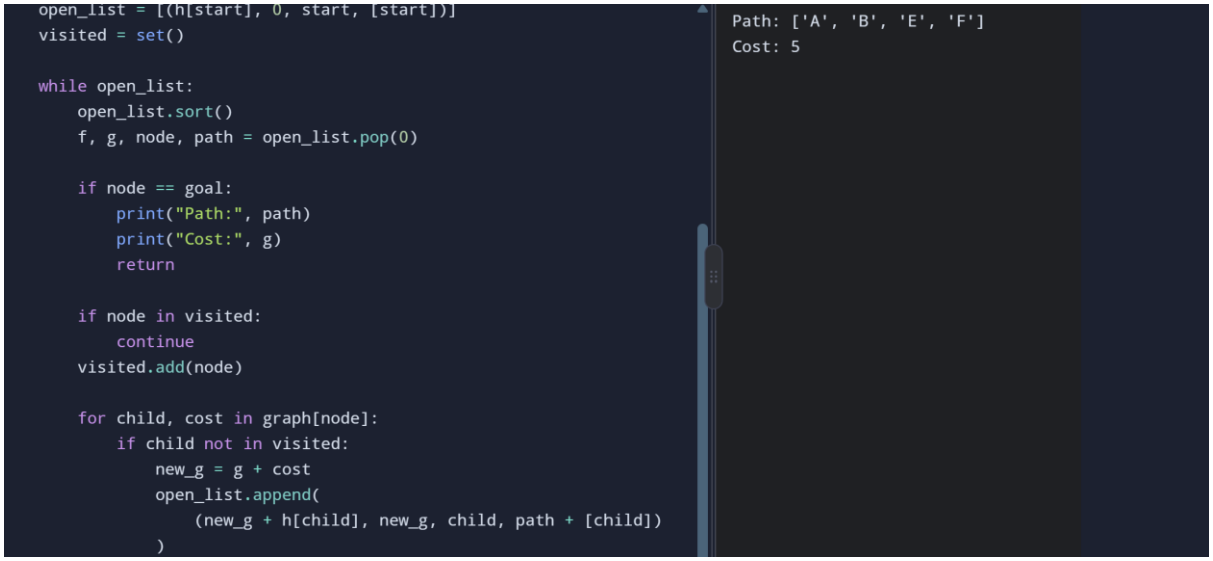
```
        open_list.sort()
        f, g, node, path = open_list.pop(0)

        if node == goal:
            print("Path:", path)
            print("Cost:", g)
            return

        if node in visited:
            continue
        visited.add(node)

        for child, cost in graph[node]:
            if child not in visited:
                new_g = g + cost
                open_list.append(
                    (new_g + h[child], new_g, child, path + [child])
                )

astar('A', 'F')
```



4

```
graph = {
   'S': [('A', 2), ('B', 3)],
   'A': [('C', 2), ('D', 4)],
   'B': [('E', 2)],
   'C': [('Z', 4)],
   'D': [('Z', 2)],
   'E': [('Z', 3)],
   'Z': []
}

h = {'S': 6, 'A': 5, 'B': 4,
    'C': 3, 'D': 2, 'E': 2, 'Z': 0}

def astar(start, goal):
   open_list = [(h[start], 0, start, [start])]
   visited = set()
```

```
while open_list:
    open_list.sort()
    f, g, node, path = open_list.pop(0)

    if node == goal:
        print("Path:", path)
        print("Cost:", g)
        return

    if node in visited:
        continue
    visited.add(node)

    for child, cost in graph[node]:
        if child not in visited:
            new_g = g + cost
            open_list.append(
                (new_g + h[child], new_g, child, path + [child])
            )
```

astar('S', 'Z')

